IaC Provisioning for Agriculture Support Triage Agent System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	DEEPAK PATRO	EN22CS301316
2.	BHOOMIKA PATIDAR	EN22CS301270
3.	CHARU NARIYA	EN22CS301290
4.	DEV KUMAR DAS	EN22CS301321
5.	AVICHAL MISHRA	EN22CS301243

Group Name: 07D3

Project Number: AAI-31

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025–2026

Problem Statement & Objectives

1. Problem Statement

   Agriculture remains one of the most critical sectors in the economy, yet farmers often face challenges in accessing timely, accurate, and reliable information related to crop management, disease diagnosis, and government policies. Traditional advisory systems are either manual, slow, or lack scalability, leading to delayed decision-making and reduced productivity.

   At the same time, modern AI-based solutions exist but are difficult to deploy and maintain due to:

- Complex infrastructure requirements
- Manual configuration processes
- Lack of automation in deployment
- High chances of human error

  Additionally, deploying such systems repeatedly across environments (development, testing, production) becomes inefficient without proper DevOps practices.

  Therefore, there is a need for a system that not only provides intelligent agricultural assistance but also ensures:

- Automated deployment
- Scalable infrastructure
- Reliable performance
- Minimal manual intervention

2. Project Objectives

   The primary objectives of this project are:

- To design and develop an **AI-powered agriculture support assistant** capable of handling user queries related to crops, diseases, and policies
- To implement **DevOps practices** for automating the entire deployment lifecycle
- To utilize **Infrastructure as Code (IaC)** for reproducible and consistent infrastructure setup
- To ensure system **scalability and reliability** using cloud services
- To minimize human errors through automation
- To provide a seamless and user-friendly interface for interaction
- To integrate modern tools such as Docker, Terraform, and CI/CD pipelines

3. Scope of the Project

   The scope of the project includes both **application development** and **deployment automation**.

Included in Scope:

- Development of a **web-based AI assistant** using Streamlit
- Integration with **Groq API** for natural language processing
- Implementation of **AI logic using CrewAI-based modular design**
- Containerization using Docker for consistent runtime environment
- Automation of deployment using GitHub Actions (CI/CD)
- Infrastructure provisioning using Terraform
- Deployment on AWS EC2

Excluded from Scope:

- Training custom machine learning models
- Large-scale production-grade distributed systems
- Mobile application development (future scope)

  The project focuses primarily on **system integration, automation, and deployment architecture**.

Proposed Solution

1. Key Features

The proposed system offers the following key features:

- Intelligent AI-based query handling system
- Domain-specific responses related to agriculture
- User-friendly web interface built using Streamlit
- Real-time response generation using LLM (Groq API)
- Fully automated CI/CD pipeline
- Containerized application ensuring portability
- Infrastructure provisioning using Terraform
- Cloud-based deployment on AWS
- Secure storage of sensitive data (API keys using GitHub Secrets)
- Scalable architecture with potential for future expansion

2. Overall Architecture / Workflow

The system is designed using two interconnected architectural perspectives:

Functional Architecture (AI System)

This architecture defines how user queries are processed:

1. The user interacts with the system through a web browser
2. The request is handled by the Streamlit-based frontend (app.py)
3. The backend processes the query using AI logic
4. The system follows a modular approach using CrewAI concepts
5. Query is enhanced using retrieval techniques (RAG if implemented)
6. The processed query is sent to the Groq API

7. The LLM generates a response
8. The response is returned and displayed to the user

This architecture ensures:

- Efficient query processing
- Context-aware responses
- Modular system design

Deployment Architecture (DevOps Pipeline)

This architecture ensures automated deployment:

1. Developer pushes code to GitHub repository
2. CI/CD pipeline (GitHub Actions) is triggered automatically
3. Application is built and Docker image is created
4. Docker image is pushed to a container registry
5. Terraform provisions infrastructure on AWS
6. EC2 instance is created with necessary configurations
7. Docker container is deployed on EC2
8. Application becomes accessible to users

This ensures:

- Zero manual deployment
- Consistent environment setup
- Faster delivery cycles

3. Tools & Technologies Used

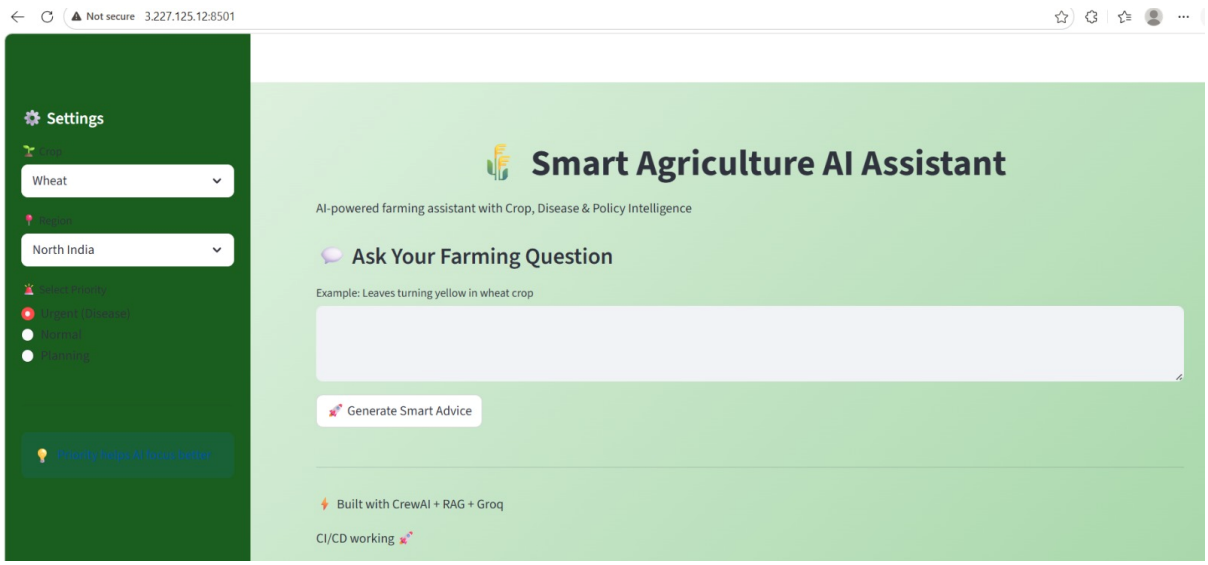
Category	Tool	Purpose
Programming	Python	Core application logic
Frontend	Streamlit	UI development
AI Framework	CrewAI	Agent-based processing
LLM API	Groq API	Response generation
Containerization	Docker	Application packaging
Version Control	GitHub	Code management
CI/CD	GitHub Actions	Automation pipeline
IaC	Terraform	Infrastructure provisioning
Cloud	AWS EC2	Hosting environment

Results & Output

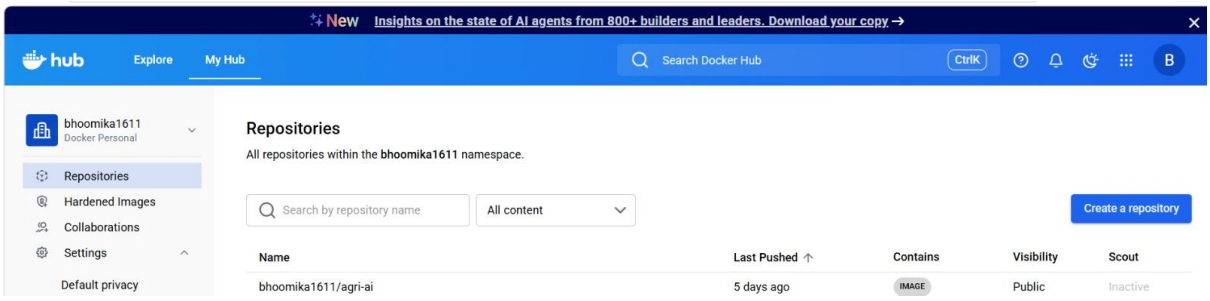
1. Screenshots / Outputs

The system produces the following outputs:

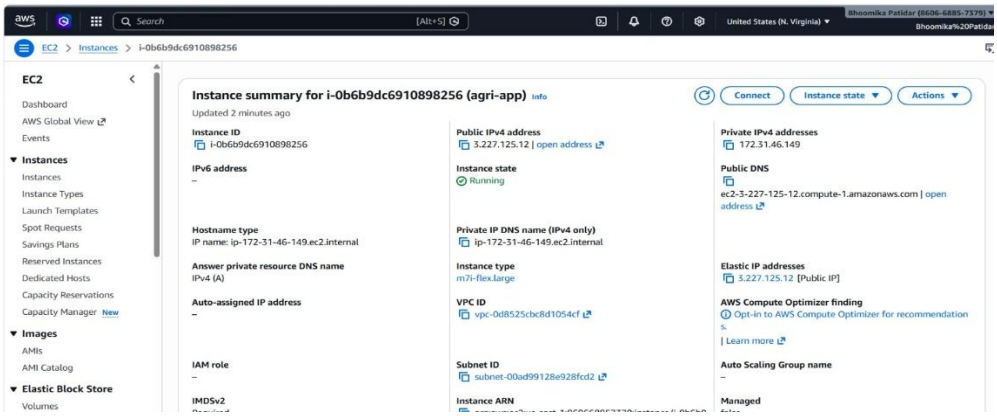
- Interactive web interface displaying user queries and responses



- AI-generated responses based on user input
- Deployment pipeline execution logs



- Running application on AWS EC2 instance



2. Reports / Dashboards / Models

- The system functions as a **real-time AI response system** rather than a static dashboard
- Outputs are generated dynamically based on user queries
- AI responses act as the primary output of the system

3. Key Outcomes

The project achieved the following outcomes:

- Successfully developed a **functional AI-based agriculture assistant**
- Implemented a **complete DevOps pipeline** for automation
- Reduced deployment time significantly
- Eliminated manual configuration errors
- Achieved consistent and reproducible deployments
- Demonstrated integration of multiple modern technologies
- Built a scalable and maintainable system architecture

Conclusion

The project successfully demonstrates the practical implementation of **DevOps principles combined with AI technologies** to build a scalable and efficient agriculture support system.

By integrating tools such as Docker, Terraform, GitHub Actions, and AWS, the system achieves:

- Automation
- Scalability
- Reliability

The use of AI APIs enables intelligent query handling, making the system useful for real-world agricultural assistance.

Overall, the project highlights the importance of combining **modern software engineering practices with intelligent systems** to solve real-world problems efficiently.

Future Scope & Enhancements

The project can be further enhanced in the following ways:

- Integration of **real-time agricultural datasets and APIs**
- Development of a **mobile application version**
- Implementation of **multi-language support for farmers**
- Advanced AI models for improved accuracy
- Migration to **Kubernetes for auto-scaling and orchestration**
- Integration of **monitoring tools (Prometheus, Grafana)**
- Implementation of **user authentication and personalization**
- Addition of voice-based interaction